

Informe de resultados

Tercer cuatrimestre

Procesamiento de Lenguaje Natural (PLN)

Mtro. Pablo Ricardo Sanchez Gomez

Jessica Melani Romero Lora

Victor Manuel Santos Martínez

10 de mayo del 2026



Tabla de contenido

1. Fundamento técnico.....	3
2. Resultados Centrales.....	7
3. Conclusiones.....	8
4. Reflexiones.....	9
5. Aplicaciones futuras y potencial metodológico.....	10

En esta tarea se desarrolló un pipeline básico de Procesamiento de Lenguaje Natural (PLN) aplicado a retroalimentación cualitativa de estudiantes focalizado en el desempeño académico de un profesor seleccionado. El corpus de entrada se compone de una base de datos compuesta por 20 comentarios abiertos sobre la clase. El objetivo de este pipeline es transformar texto en datos estructurados con categoría temática e identificar la polaridad del sentimiento, identificando neutralidad, negatividad o positividad en torno al desempeño del profesorado.

El trabajo contempla la ingesta de los datos, lectura, definición de funciones y la exportación de resultados en archivo CSV, el cual incluye (id, comentario_original, texto_limpio, tokens_utiles, palabras_frecuentes, categoria, sentimiento), esto tiene como objetivo identificar el contenido fundamental de la información.

1. Fundamento técnico

El código implementa una arquitectura secuencial modular en la que cada etapa encapsula una responsabilidad única y opera sobre la salida de la etapa precedente. Este diseño favorece la trazabilidad del procesamiento y permite sustituir o extender cualquier componente sin afectar al resto del sistema.

La primera etapa, **carga y lectura**, ingesta el archivo de texto plano en codificación UTF-8 y produce una lista de cadenas, una por comentario. La segunda etapa, **normalización**, aplica tres transformaciones sucesivas: conversión a minúsculas, eliminación de marcas diacríticas mediante descomposición Unicode (NFD) y remoción de signos de puntuación mediante expresiones regulares. El resultado es una cadena canónica que elimina la variabilidad ortográfica superficial sin alterar el contenido léxico. La tercera etapa, **tokenización**, divide cada cadena normalizada en unidades discretas mediante separación por espacio (*whitespace tokenization*), produciendo una lista de tokens candidatos. La cuarta etapa, **filtrado**, descarta los tokens que pertenecen al conjunto de stopwords definido manualmente o que no superan una longitud mínima de dos caracteres, conservando únicamente los tokens con potencial carga semántica. La quinta etapa, **clasificación**, asigna a cada comentario una etiqueta de sentimiento y una categoría temática mediante el método que se describe a continuación.

El enfoque de clasificación adoptado es **basado en léxico** (*lexicon-based*), lo que implica que no se emplea ningún modelo estadístico entrenado sobre datos etiquetados. La polaridad de sentimiento se determina por conteo diferencial: se

cuantifican los tokens del comentario que pertenecen al conjunto de palabras positivas y los que pertenecen al conjunto de palabras negativas, previamente definidos de forma manual. La comparación entre ambos conteos produce una de cuatro clases de salida: *positivo* (predominan tokens positivos), *negativo* (predominan tokens negativos), *mixto* (ambos conjuntos tienen representación equivalente) y *neutral* (ningún conjunto tiene representación). La clasificación temática sigue el mismo principio: cada comentario se asigna a la categoría —*docente*, *contenido* o *plataforma*— cuyo diccionario léxico acumula el mayor número de coincidencias con los tokens útiles del comentario.

Primera y segunda etapa de diseño de pipeline:

```
1. Importación de librerías

import sys
print(f"Python version: {sys.version}")

from pathlib import Path
import csv
import re
import unicodedata
from collections import Counter
import pandas as pd

[1]
Python version: 3.12.6 (main, Oct 11 2025, 10:55:08) [Clang 17.0.0 (clang-1700.3.19.1)]

2. Definición de diccionarios (stopwords, palabras positivas/negativas, categorías)

ARCHIVO_ENTRADA = Path("comentarios_clase1.txt")
ARCHIVO_SALIDA = Path("resultados_clase1.csv")

STOPWORDS = {
    "el", "la", "los", "las", "un", "una", "unos", "unas",
    "de", "del", "y", "o", "pero", "aunque", "a", "en", "con",
    "por", "para", "me", "mi", "mis", "es", "fue", "muy", "que",
    "se", "al", "lo", "no", "su", "sus", "tuvo", "durante"
}

PALABRAS_POSITIVAS = {
    "excelente", "bien", "buena", "bueno", "interesante", "util", "gusto",
    "claro", "claridad", "paciencia", "relevante", "ayudo", "facil", "organizado"
}

PALABRAS_NEGATIVAS = {
    "demasiadas", "rapido", "faltaron", "fallo", "confusos", "confuso",
    "poca", "lenta", "mal", "problema", "cayo", "demasiado", "no", "permitio"
}

PALABRAS_CATEGORIA = {
    "docente": {"profesor", "docente", "explica", "explicacion", "dudas", "retroalimentacion", "calificar"},
    "contenido": {"clase", "materia", "contenido", "conceptos", "tema", "temas", "teoria", "practica", "prácticas", "actividad", "material"},
    "plataforma": {"plataforma", "sistema", "archivo", "entrega", "subir", "cayo"},
}

print("Diccionarios cargados correctamente.")

[2]
Diccionarios cargados correctamente.
```

Tercera etapa de diseño de pipeline: definición de funciones principales

3. Funciones del pipeline

```
def cargar_comentarios(ruta: Path) -> list[str]:
    if not ruta.exists():
        raise FileNotFoundError(f"No existe el archivo de entrada: {ruta}")
    comentarios = [linea.strip() for linea in ruta.read_text(encoding="utf-8").splitlines()]
    return [comentario for comentario in comentarios if comentario]

def quitar_acentos(texto: str) -> str:
    normalizado = unicodedata.normalize("NFD", texto)
    return "".join(c for c in normalizado if unicodedata.category(c) != "Mn")

def limpiar_texto(texto: str) -> str:
    texto = texto.lower()
    texto = quitar_acentos(texto)
    texto = re.sub(r"[a-z0-9s]", " ", texto)
    texto = re.sub(r"\s+", " ", texto).strip()
    return texto

def tokenizar(texto_limpio: str) -> list[str]:
    return texto_limpio.split()

def quitar_stopwords(tokens: list[str]) -> list[str]:
    return [token for token in tokens if token not in STOPWORDS and len(token) > 2]

def clasificar_sentimiento(tokens: list[str]) -> str:
    positivos = sum(1 for token in tokens if token in PALABRAS_POSITIVAS)
    negativos = sum(1 for token in tokens if token in PALABRAS_NEGATIVAS)
    if positivos > 0 and negativos > 0:
        return "mixto"
    if positivos > negativos:
        return "positivo"
    if negativos > positivos:
        return "negativo"
    return "neutral"

def clasificar_categoria(tokens: list[str]) -> str:
    puntajes = {}
    for categoria, palabras in PALABRAS_CATEGORIA.items():
        puntajes[categoria] = sum(1 for token in tokens if token in palabras)
    mejor_categoria, mejor_puntaje = max(puntajes.items(), key=lambda item: item[1])
    return mejor_categoria if mejor_puntaje > 0 else "sin_categoria"
```

```
def clasificar_categoria(tokens: list[str]) -> str:
    puntajes = {}
    for categoria, palabras in PALABRAS_CATEGORIA.items():
        puntajes[categoria] = sum(1 for token in tokens if token in palabras)
    mejor_categoria, mejor_puntaje = max(puntajes.items(), key=lambda item: item[1])
    return mejor_categoria if mejor_puntaje > 0 else "sin_categoria"

def analizar_comentarios(comentarios: list[str]) -> list[dict[str, str]]:
    resultados = []
    for i, comentario in enumerate(comentarios, start=1):
        limpio = limpiar_texto(comentario)
        tokens = tokenizar(limpio)
        tokens utiles = quitar_stopwords(tokens)
        frecuencia = Counter(tokens_uitles)
        resultados.append({
            "id": str(i),
            "comentario_original": comentario,
            "texto_limpio": limpio,
            "tokens_uitles": ", ".join(tokens_uitles),
            "palabras_frecuentes": ", ".join([f"{p}:{c}" for p, c in frecuencia.most_common(5)]),
            "categoria": clasificar_categoria(tokens_uitles),
            "sentimiento": clasificar_sentimiento(tokens_uitles),
        })
    return resultados

print("Funciones definidas correctamente.")
```

Funciones definidas correctamente.

Cuarta etapa de diseño de pipeline:

4. Carga y procesamiento de comentarios

```
> comentarios = cargar_comentarios(ARCHIVO_ENTRADA)
print(f"Se cargaron {len(comentarios)} comentarios.\n")

for i, c in enumerate(comentarios, 1):
    print(f" {i:2d}. {c}")
```

[4]

... Se cargaron 20 comentarios.

1. El profesor explica muy bien, pero deja demasiadas tareas.
2. La clase es interesante, aunque a veces va muy rápido.
3. No entendí bien los temas porque faltaron ejemplos prácticos.
4. Excelente materia, me gustó trabajar con casos reales.
5. El docente domina el tema, pero debería organizar mejor los tiempos.
6. La plataforma falló varias veces durante la entrega.
7. Me gustaría que hubiera más prácticas y menos teoría.
8. El contenido es útil, pero algunos conceptos fueron confusos.
9. El profesor resolvió dudas con paciencia y claridad.
10. La actividad fue buena, aunque el tiempo no alcanzó.
11. No me gustó que la explicación fuera demasiado rápida.
12. La plataforma no permitió subir el archivo a tiempo.
13. El tema fue relevante para mi trabajo profesional.
14. La clase tuvo mucha teoría y poca aplicación.
15. La práctica ayudó a entender mejor el concepto.
16. El docente fue claro, pero faltó retroalimentación.
17. El material estaba organizado y fue fácil de seguir.
18. El sistema se cayó durante la entrega final.
19. Me pareció excelente trabajar con ejemplos reales.
20. El contenido fue confuso al inicio, pero la práctica ayudó.

```

resultados = analizar_comentarios(comentarios)

for fila in resultados:
    print("=" * 90)
    print(f"ID: {fila['id']}")
    print(f"Comentario : {fila['comentario_original']}")
    print(f"Limpio      : {fila['texto_limpio']}")
    print(f"Tokens       : {fila['tokens utiles']}")
    print(f"Categoría    : {fila['categoria']}")
    print(f"Sentimiento: {fila['sentimiento']}")
print("=" * 90)

```

```

=====
ID: 1
Comentario : El profesor explica muy bien, pero deja demasiadas tareas.
Limpio      : el profesor explica muy bien pero deja demasiadas tareas
Tokens      : profesor, explica, bien, deja, demasiadas, tareas
Categoría   : docente
Sentimiento: mixto
=====
ID: 2
Comentario : La clase es interesante, aunque a veces va muy rápido.
Limpio      : la clase es interesante aunque a veces va muy rapido
Tokens      : clase, interesante, veces, rapido
Categoría   : contenido
Sentimiento: mixto
=====
ID: 3
Comentario : No entendí bien los temas porque faltaron ejemplos prácticos.
Limpio      : no entendi bien los temas porque faltaron ejemplos practicos
Tokens      : entendi, bien, temas, porque, faltaron, ejemplos, practicos
Categoría   : contenido
Sentimiento: mixto
=====
ID: 4
Comentario : Excelente materia, me gustó trabajar con casos reales.
Limpio      : excelente materia me gusto trabajar con casos reales
...
Tokens      : contenido, confuso, inicio, practica, ayuda
Categoría   : contenido
Sentimiento: mixto
=====

```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

2. Resultados Centrales

De los 20 comentarios procesados, los resultados siguen una distribución por sentimiento en donde predominan los comentarios clasificados como mixtos, aquellos con señales positivas y negativas simultáneas, seguidos de positivo y negativo; pocos como neutral.

La distribución por categoría señala que la mayoría se clasificó bajo contenido (referencias a materias, teoría, práctica, actividades) dentro de la base de datos; le sigue docente (explicaciones, dudas, retroalimentación); y plataforma concentra los comentarios negativos más directos (fallos al subir archivos, caída del sistema entre otros).

```
5. Resultados en tabla y guardado en CSV

df = pd.DataFrame(resultados)
display(df[["id", "comentario_original", "categoria", "sentimiento"]])

...

# Guardar resultados en CSV
campos = ["id", "comentario_original", "texto_limpio", "tokens utiles",
          "palabras_frecuentes", "categoria", "sentimiento"]

with ARCHIVO_SALIDA.open("w", encoding="utf-8-sig", newline="") as archivo:
    escritor = csv.DictWriter(archivo, fieldnames=campos)
    escritor.writeheader()
    escritor.writerows(resultados)

print(f"Resultados guardados en: {ARCHIVO_SALIDA.resolve()}")

... Resultados guardados en: /Users/victor/Library/Mobile Documents/com-apple~CloudDocs/Maestría UPMH/Cuatrimestre 3/Procesamiento De Lenguaje Natural/Tarea 1/resultados_clase1.csv
```

El hallazgo metodológico crítico identifica que la palabra "no" estaba incluida simultáneamente en STOPWORDS (donde se elimina) y en PALABRAS_NEGATIVAS (donde debería contribuir). Sumado al filtro `len(token) > 2`, esto generaba un doble descarte de la negación, causando comentarios negativos como "No me gustó..." se clasificaron incorrectamente como mixtos o neutrales. La corrección implicó retirar "no" de STOPWORDS y ajustar el filtro a `len >= 2`, esto se realizó manualmente manteniendo la coherencia lógica.

```
5. Resultados en tabla y guardado en CSV

df = pd.DataFrame(resultados)
display(df[["id", "comentario_original", "categoria", "sentimiento"]])

...

# Guardar resultados en CSV
campos = ["id", "comentario_original", "texto_limpio", "tokens utiles",
          "palabras_frecuentes", "categoria", "sentimiento"]

with ARCHIVO_SALIDA.open("w", encoding="utf-8-sig", newline="") as archivo:
    escritor = csv.DictWriter(archivo, fieldnames=campos)
    escritor.writeheader()
    escritor.writerows(resultados)

print(f"Resultados guardados en: {ARCHIVO_SALIDA.resolve()}")

... Resultados guardados en: /Users/victor/Library/Mobile Documents/com-apple~CloudDocs/Maestría UPMH/Cuatrimestre 3/Procesamiento De Lenguaje Natural/Tarea 1/resultados_clase1.csv
```


3. Conclusiones

El pipeline diseñado cumple cuatro funciones centrales relevantes donde destaca la estandarización lingüística, es decir, la normalización (acentos, capitalización, puntuación) enfocado en la reducción de variabilidad ortográfica del español coloquial, condición necesaria para cualquier análisis léxico reproducible.

De igual manera, se efectuó un proceso de reducción de ruido en donde se realizó la eliminación de stopwords mediante un filtrado de palabras sin carga semántica, concentrando el análisis en tokens informativos para el CSV resultante. Esto es particularmente importante en español, donde artículos y preposiciones son altamente frecuentes y parte de la estructura básica del idioma. Bajo esta ejecución, el etiquetado estructurado convierte texto no estructurado en registros tabulares categorizables, habilitando análisis cuantitativo sobre corpus cualitativos, el cual constituye el objetivo principal de esta práctica.

Durante el procesamiento de la información se llevó a cabo la identificación de conflictos lógicos en preprocesamiento diversos, por ejemplo, el error documentado en la sección 6 ilustra un problema real en diseño de pipelines léxicos, en donde la coherencia entre etapas de filtrado y etapas de clasificación no es automática y debe verificarse explícita y manualmente.

4. Reflexiones

El pipeline implementado cumple funciones técnicas que trascienden el objetivo inmediato de la tarea, siendo relevantes tanto para el dominio educativo como para el PLN aplicado en general.

La primera función es la estandarización lingüística. La fase de normalización —que abarca la eliminación de acentos, la conversión a minúsculas y la remoción de puntuación mediante expresiones regulares— reduce la variabilidad ortográfica inherente al español coloquial. Sin esta etapa, tokens semánticamente equivalentes como *"Excelente"*, *"excelente"* y *"excelente,"* serían tratados como entidades léxicas distintas, comprometiendo la reproducibilidad de cualquier análisis posterior.

La segunda función es la reducción de dimensionalidad léxica. La eliminación de stopwords descarta palabras funcionales de alta frecuencia pero escaso contenido semántico: artículos, preposiciones y conjunciones. En español, estas palabras

pueden representar entre el 40 y el 50% del vocabulario de un texto breve; su remoción concentra el procesamiento en los tokens informativamente relevantes y reduce el espacio de representación sin pérdida significativa de contenido.

La tercera función es la producción de datos estructurados. El pipeline convierte texto libre en registros tabulares etiquetados, condición necesaria para habilitar análisis cuantitativo sobre corpus cualitativos. Esta transformación es el punto de entrada a cualquier flujo de trabajo de minería de opiniones a mayor escala, desde reportes descriptivos hasta el entrenamiento de modelos supervisados.

La cuarta función, de naturaleza metodológica, es la explicitación de conflictos en el diseño del preprocesamiento. El error identificado y documentado en la sección 3.2 —el doble filtrado del token *"no"*— ilustra un principio crítico del diseño de pipelines léxicos: la coherencia entre las etapas de filtrado y las etapas de clasificación no es automática. Un token eliminado en la fase de preprocesamiento nunca podrá contribuir en fases de inferencia posteriores, y esta condición puede degradar silenciosamente la calidad del sistema sin producir ningún error de ejecución visible. La práctica de documentar y corregir este tipo de inconsistencias es fundamental para garantizar la validez y reproducibilidad del procesamiento.

5. Aplicaciones futuras y potencial metodológico

El sistema implementado establece una base funcional sobre la cual pueden aplicarse mejoras progresivas. Sus limitaciones actuales son reconocibles y cada una tiene una extensión metodológica correspondiente.

La tokenización por espacio, que no resuelve correctamente unidades multipalabra ni contracciones, puede sustituirse por tokenizadores lingüísticos especializados como spaCy con el modelo `es_core_news_sm` o NLTK, que incorporan reglas morfológicas del español.

La ausencia de lematización —por la cual *"gustó"* y *"gusto"* son tratados como tipos léxicos distintos— puede corregirse mediante el lematizador de spaCy o el algoritmo de stemming SnowballStemmer en su variante para español.

Mientras que, los diccionarios manuales de cobertura reducida pueden reemplazarse por léxicos de sentimiento preexistentes y validados, como AFINN-es, SentiWordNet o ML-SentiCon.

La evaluación de tokens en aislamiento, que impide capturar negaciones sintácticas, puede extenderse hacia el análisis de bigramas o n-gramas, capturando construcciones como *"no me gustó"* o *"no entendí"* como unidades semánticas. La clasificación mono-etiqueta por categoría puede transformarse en clasificación multi-label mediante la introducción de umbrales de puntaje diferenciados. Finalmente, la escala reducida del corpus y el enfoque basado en reglas pueden superarse mediante modelos supervisados —Naive Bayes, SVM— o mediante el ajuste fino (*fine-tuning*) de BETO, la versión de BERT entrenada sobre corpus en español, para conjuntos de datos de mayor volumen.

En todos los casos, la incorporación de métricas formales de evaluación —accuracy, F1-score por clase, matriz de confusión— resulta indispensable para cuantificar el desempeño real del sistema frente a etiquetas de referencia. El paso más significativo hacia un sistema de mayor capacidad práctica es la adopción de embeddings contextuales, como BETO o RoBERTa-base-bne.

A diferencia de los diccionarios léxicos, estos modelos producen representaciones del significado de cada token en función de su contexto oracional, lo que permite resolver de forma nativa tanto el problema de la negación sintáctica como el de la polisemia, sin necesidad de reglas manuales adicionales.